

39. GPU 候補の厳密検証

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	scripts/validate_gpu_upper_policy_candidates.py
種別	Python
区分	planning validation
役割	GPU 探索で得た speed policy 候補を CPU 側 exact replay と gate で再判定する。
起動文脈	GPU acceptance の中心。

このファイルを読む理由

GPU 探索で得た speed policy 候補を CPU 側 exact replay と gate で再判定する。

- numerical match、mission feasibility、gate pass を確認する。

呼び出し関係

どこから呼ばれるか

- GPU acceptance pipeline
- 手動検証

次に読むと理解が進むファイル

- scripts/run_upper_mesh_convergence.py
- scripts/solar_sim.py

同一リポジトリ内の主要依存

- scripts/run_upper_mesh_convergence.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

Replay every GPU policy in the exact 1 Hz simulator and select the fastest feasible one.

代表コード断片

```
ROOT = Path(__file__).resolve().parents[1]

def resolve_result_path(raw: str) -> Path:
    path = Path(str(raw))
    return path if path.is_absolute() else (ROOT / path).resolve()

def resolve_profile_asset(profile_path: Path, raw: str) -> Path:
    path = Path(str(raw))
    return path if path.is_absolute() else (profile_path.parent / path).resolve()

def evaluate_event_timing(detail: pd.DataFrame, cfg: dict, profile_path: Path) -> dict:
    """Check official control-stop closing times and the absolute finish deadline."""
    paths = cfg.get("paths", {}) or {}
    simulation = cfg.get("simulation", {}) or {}
    stop_path = resolve_profile_asset(profile_path, paths.get("stop_yaml", ""))
    stop_cfg = (
        yaml.safe_load(stop_path.read_text(encoding="utf-8-sig")) or {}
        if stop_path.is_file()
        else {}
    )
```

主要構造の読み取り

主要関数は `resolve_result_path`, `resolve_profile_asset`, `evaluate_event_timing`, `exact_replay_signature`, `inspect_prediction_mesh`, `prediction_execution_soc_errors`, `evaluate_soc_guard_intervention`, `main`。CLI 引数宣言は 8 件。

主要クラス・関数

行	種別	名前	補足
26	function	<code>resolve_result_path</code>	args: raw
31	function	<code>resolve_profile_asset</code>	args: profile_path, raw
36	function	<code>evaluate_event_timing</code>	args: detail, cfg, profile_path
112	function	<code>exact_replay_signature</code>	args: profile, policy
137	function	<code>inspect_prediction_mesh</code>	args: manifest, manifest_path
199	function	<code>prediction_execution_soc_errors</code>	args: manifest
228	function	<code>evaluate_soc_guard_intervention</code>	args: detail, manifest
249	function	<code>main</code>	

ファイルを上から読んだときの定義順

行	内容
17	例外処理を伴う <code>try</code> ブロックを実行する。
23	<code>ROOT</code> に <code>Path(__file__).resolve().parents[1]</code> の結果を代入する。

26 関数 `resolve_result_path` を定義する。
 31 関数 `resolve_profile_asset` を定義する。
 36 関数 `evaluate_event_timing` を定義する。
 112 関数 `exact_replay_signature` を定義する。
 137 関数 `inspect_prediction_mesh` を定義する。
 199 関数 `prediction_execution_soc_errors` を定義する。
 228 関数 `evaluate_soc_guard_intervention` を定義する。
 249 関数 `main` を定義する。
 476 条件 `__name__ == '__main__'` を判定し、真なら内部処理を行う。

import 群の詳細

行	import 文	このファイルで読む理由
4	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
6	<code>importargparse</code>	CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L250。
7	<code>importjson</code>	manifest、checkpoint、UDP payload をやり取りするため。このファイル内での主な使用位置は L309, L340, L470, L472。
8	<code>importmath</code>	clip、sqrt、sin/cos、有限判定など数値ロジックに使うため。このファイル内での主な使用位置は L148, L205, L210。
9	<code>importshutil</code>	shutil モジュールを利用するため。このファイル内での主な使用位置は L458。
10	<code>importsubprocess</code>	git 情報取得や外部コマンド実行を行うため。このファイル内での主な使用位置は L318, L322。
11	<code>importsys</code>	sys モジュールを利用するため。このファイル内での主な使用位置は L319。
12	<code>frompathlibimportPath</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L23, L26, L27, L31, L32, L36, L112, L123, ...。
14	<code>importpandasaspd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L36, L48, L49, L50, L63, L64, L88, L89, ...。
15	<code>importyaml</code>	profile、stop、schedule、summary YAML を読み書きするため。このファイル内での主な使用位置は L42, L127, L272, L297。
18	<code>fromscripts.run_upper_ mesh_convergenceimportfile_ sha256,materialize_profile</code>	upper mesh 収束確認から file_sha256, materialize_profile を読み込み、このファイルの内部処理を分担させるため。実体ファイルは scripts/run_upper_mesh_convergence.py。このファイル内での主な使用位置は L125, L126, L133, L282。
20	<code>fromrun_upper_mesh_ convergenceimportfile_ sha256,materialize_profile</code>	run_upper_mesh_convergence から file_sha256, materialize_profile を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L125, L126, L133, L282。

関数・クラスを上から順に解説

L26 関数 `resolve_result_path`

- 定義: `resolve_result_path(raw)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `is_absolute`, `resolve`, `str`
- 戻り値の要点: `path if path.is_absolute() else (ROOT / path).resolve()`

1. `path` に `Path(str(raw))` の結果を代入する。
2. `path if path.is_absolute() else (ROOT / path).resolve()` を返す。

L31 関数 `resolve_profile_asset`

- 定義: `resolve_profile_asset(profile_path, raw)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `is_absolute`, `resolve`, `str`
- 戻り値の要点: `path if path.is_absolute() else (profile_path.parent / path).resolve()`

1. `path` に `Path(str(raw))` の結果を代入する。
2. `path if path.is_absolute() else (profile_path.parent / path).resolve()` を返す。

L36 関数 `evaluate_event_timing`

- 定義: `evaluate_event_timing(detail, cfg, profile_path)`
- docstring: Check official control-stop closing times and the absolute finish deadline.
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `Timestamp`, `all`, `append`, `bool`, `dropna`, `float`, `get`, `is_file`, `isna`, `isoformat`, `max`
- 戻り値の要点: `{ 'control_stop_windows_pass': bool(all_stops_observed and all((row['passed'] for row in stop_results))), 'finish_deadline_pass': bool(deadline_late_sec <= 1.0), 'max_control_stop_late_sec': max_late_sec, 'finish_deadline_late_sec': deadline_late_sec, 'finish_time_utc': finish_time.isoformat() if not pd.isna(finish_time) else '', 'race_deadline_utc': deadline.isoformat() if deadline_raw else '', 'control_stops': stop_results }`

1. `paths` に `cfg.get('paths', {})` or `{}` の結果を代入する。
2. `simulation` に `cfg.get('simulation', {})` or `{}` の結果を代入する。
3. `stop_path` に `resolve_profile_asset(profile_path, paths.get('stop_yaml', ''))` の結果を代入する。
4. `stop_cfg` に `yaml.safe_load(stop_path.read_text(encoding='utf-8-sig'))` or `{}` if `stop_path.is_file()` else `{}` の結果を代入する。
5. `distance_col` に `'s_end_km' if 's_end_km' in detail else 's_km'` の結果を代入する。

6. `time_col` に `'time_end_utc'` if `'time_end_utc'` in detail else `'time_utc'` の結果を代入する。
7. `distance` に `pd.to_numeric(detail.get(distance_col), errors='coerce')` の結果を代入する。
8. `times` に `pd.to_datetime(detail.get(time_col), errors='coerce', utc=True, format='mixed')` の結果を代入する。
9. `ordered` に `pd.DataFrame({'s_km': distance, 'time': times}).dropna().sort_values('time')` の結果を代入する。
10. `stop_results` に `[]` の結果を代入する。

L112 関数 `exact_replay_signature`

- 定義: `exact_replay_signature(profile, policy)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `Path`, `encode`, `file_sha256`, `get`, `hexdigest`, `is_file`, `isinstance`, `join`, `read_text`, `resolve`, `resolve_profile_asset`, `safe_load`
 - 戻り値の要点: `hashlib.sha256(payload.encode('ascii')).hexdigest()`
1. Import 文を実行する。
 2. `dependencies` に `(ROOT / 'scripts' / 'solar_sim.py', ROOT / 'mpc_solarcar' / 'upper_horizon.py', ROOT / 'mpc_solarcar' / 'upper_solver.py', ROOT / 'mpc_solarcar' / 'upper_policy.py', ROOT / 'mpc_solarcar' / 'upper_cost.py', ROOT / 'mpc_solarcar' / 'model.py', ROOT / 'mpc_solarcar' / 'signal_utils.py', Path(__file__).resolve())` の結果を代入する。
 3. `payload` に `file_sha256(profile) + file_sha256(policy)` の結果を代入する。
 4. `payload` を `Add` で更新する。
 5. `cfg` に `yaml.safe_load(profile.read_text(encoding='utf-8-sig')) or {}` の結果を代入する。
 6. `(cfg.get('paths', {})) or {}` の `values()` を順に走査し、各要素を `raw` に入れて処理する。
 7. `hashlib.sha256(payload.encode('ascii')).hexdigest()` を返す。

L137 関数 `inspect_prediction_mesh`

- 定義: `inspect_prediction_mesh(manifest, manifest_path, requested_ds_km, start_s_km, race_km)`
 - docstring: Verify that the selected prediction trace used the requested distance mesh.
 - このブロックが直接呼ぶ主な関数・メソッド: `Path`, `abs`, `bool`, `ceil`, `dropna`, `float`, `get`, `int`, `is_file`, `len`, `list`, `max`
 - 戻り値の要点: `result / result / result / result`
1. `requested_ds_km` に `float(requested_ds_km)` の結果を代入する。
 2. `remaining_km` に `max(0.0, float(race_km) - float(start_s_km))` の結果を代入する。
 3. `expected_min_steps` に `int(math.ceil(remaining_km / requested_ds_km - 1e-09))` の結果を代入する。
 4. `result` に `{'valid': False, 'expected_min_steps': expected_min_steps, 'prediction_steps': 0, 'trace_drive_steps': 0, 'max_trace_ds_km': float('nan'), 'trace_terminal_km': float('nan')}` の結果を代入する。
 5. `diagnostics` に `list(manifest.get('upper_solver_diagnostics', []) or [])` の結果を代入する。

6. 条件 `not diagnostics` を判定し、真なら内部処理を行う。
7. `diagnostic` に `diagnostics[0]` の結果を代入する。
8. `result['prediction_steps']` に `int(diagnostic.get('prediction_steps', 0) or 0)` の結果を代入する。
9. `raw_trace` に `str(diagnostic.get('selected_prediction_trace_csv', '') or '')` の結果を代入する。
10. 条件 `not raw_trace` を判定し、真なら内部処理を行う。

L199 関数 `prediction_execution_soc_errors`

- 定義: `prediction_execution_soc_errors(manifest)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `append`, `dict`, `float`, `get`, `int`, `isfinite`, `list`
 - 戻り値の要点: `{'initial_prediction_soc': initial_prediction, 'latest_nontrivial_prediction_soc': latest_prediction, 'initial_error': final_soc - initial_prediction, 'latest_nontrivial_error': final_soc - latest_prediction} / {'initial_prediction_soc': float('nan'), 'latest_nontrivial_prediction_soc': float('nan'), 'initial_error': float('inf'), 'latest_nontrivial_error': float('inf')}`
1. `diagnostics` に `list(manifest.get('upper_solver_diagnostics', []) or [])` の結果を代入する。
 2. `predictions` に `[]` の結果を代入する。
 3. `diagnostics` を順に走査し、各要素を `diagnostic` に入れて処理する。
 4. `final_soc` に `float(manifest.get('final_soc', float('nan')))` の結果を代入する。
 5. 条件 `not predictions or not math.isfinite(final_soc)` を判定し、真なら内部処理を行う。
 6. `initial_prediction` に `predictions[0][1]` の結果を代入する。
 7. `nontrivial` に `[soc for steps, soc in predictions if steps > 1]` の結果を代入する。
 8. `latest_prediction` に `nontrivial[-1] if nontrivial else predictions[-1][1]` の結果を代入する。
 9. `{'initial_prediction_soc': initial_prediction, 'latest_nontrivial_prediction_soc': latest_prediction, 'initial_error': final_soc - initial_prediction, 'latest_nontrivial_error': final_soc - latest_prediction}` を返す。

L228 関数 `evaluate_soc_guard_intervention`

- 定義: `evaluate_soc_guard_intervention(detail, manifest)`
 - docstring: `Reject policies that only finish because the execution safety guard intervened.`
 - このブロックが直接呼ぶ主な関数・メソッド: `astype`, `bool`, `fillna`, `float`, `get`, `int`, `isin`, `lower`, `strip`, `sum`, `to_numeric`
 - 戻り値の要点: `{'intervention_rows': intervention_rows, 'intervention_sec': intervention_sec, 'passed': bool(intervention_rows == 0 and intervention_sec <= 1e-09)}`
1. 条件 `'soc_guard_intervened' in detail.columns` を判定し、真なら内部処理を行う。
 2. `{'intervention_rows': intervention_rows, 'intervention_sec': intervention_sec, 'passed': bool(intervention_rows == 0 and intervention_sec <= 1e-09)}` を返す。

L249 関数 main

- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ArgumentParser`, `DataFrame`, `FileNotFoundError`, `Path`, `abs`, `add_argument`, `all`, `append`, `bool`, `copy2`, `dumps`, `evaluate_event_timing`
- 戻り値の要点: `0 if selection['selected'] else 2`

1. `parser` に `argparse.ArgumentParser(description=__doc__)` の結果を代入する。
2. `parser.add_argument(...)` を実行する。
3. `parser.add_argument(...)` を実行する。
4. `parser.add_argument(...)` を実行する。
5. `parser.add_argument(...)` を実行する。
6. `parser.add_argument(...)` を実行する。
7. `parser.add_argument(...)` を実行する。
8. `parser.add_argument(...)` を実行する。
9. `parser.add_argument(...)` を実行する。
10. `args` に `parser.parse_args()` の結果を代入する。

CLI 引数

行	引数
251	<code>--profile</code>
252	<code>--campaign-dir</code>
253	<code>--output-dir</code>
254	<code>--stage</code>
255	<code>--seed-label</code>
260	<code>--integration-ds-km</code>
261	<code>--control-ds-km</code>
262	<code>--no-resume</code>

処理の流れ

1. CLI 引数を解釈し、`main()` から処理を起動する。

このファイルの位置づけ

この資料群では、`scripts/validate_gpu_upper_policy_candidates.py` を **planning validation** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。